PL/SQL PROGRAMMING

TABLE CREATION:

```
SQL> CREATE TABLE Customers (
  2      CustomerID NUMBER PRIMARY KEY,
  3      CustomerName VARCHAR2(50),
  4      Age NUMBER,
  5      Balance NUMBER,
  6      IsVIP VARCHAR2(5)
  7  );

Table created.

SQL>
SQL> CREATE TABLE Loans (
  2      LoanID NUMBER PRIMARY KEY,
  3      CustomerID NUMBER,
  4      InterestRate NUMBER,
  5      DueDate DATE
  6  );

Table created.

SQL>
SQL> CREATE TABLE Accounts (
  2      AccountID NUMBER PRIMARY KEY,
  3      Balance NUMBER
  4  );

Table created.

SQL>
SQL> CREATE TABLE Employees (
  2      EmployeeID NUMBER PRIMARY KEY,
  3      EmployeeName VARCHAR2(50),
  4      Department VARCHAR2(30),
  5      Salary NUMBER
  6  );

Table created.
```

INSERT VALUES:

```
SQL> INSERT INTO Customers VALUES (1,'Sai',65,15000,'FALSE');

1 row created.

SQL> INSERT INTO Customers VALUES (2,'Ravi',45,8000,'FALSE');

1 row created.

SQL> INSERT INTO Customers VALUES (3,'Priya',70,20000,'FALSE');

1 row created.

SQL>
SQL> INSERT INTO Loans VALUES (101,1,10,SYSDATE+20);

1 row created.

SQL> INSERT INTO Loans VALUES (102,2,12,SYSDATE+50);

1 row created.

SQL> INSERT INTO Loans VALUES (103,3,11,SYSDATE+10);

1 row created.

SQL>
SQL> COMMIT;

Commit complete.
```

DISPLAY TABLES:

```
SQL> SELECT * FROM Customers;

CUSTOMERID CUSTOMERNAME                                         AGE    BALANCE ISVIP
---------- -------------------------------------------------- ---------- ---------- -----
         1 Sai                                                 65      15000 FALSE
         2 Ravi                                                45       8000 FALSE
         3 Priya                                               70      20000 FALSE

SQL> SELECT * FROM Loans;

    LOANID CUSTOMERID INTERESTRATE DUEDATE
---------- ---------- ------------ ---------
       101          1           10 12-JUL-26
       102          2           12 11-AUG-26
       103          3           11 02-JUL-26
```

Exercise 1: Control Structures

Scenario 1: The bank wants to apply a discount to loan interest rates for customers above 60 years old.
Question: Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.

```
SQL> BEGIN
  2
  3      FOR customer_rec IN
  4      (
  5          SELECT CustomerID, Age
  6          FROM Customers
  7      )
  8      LOOP
  9
 10          IF customer_rec.Age > 60 THEN
 11
 12              UPDATE Loans
 13              SET InterestRate = InterestRate - 1
 14              WHERE CustomerID = customer_rec.CustomerID;
 15
 16          END IF;
 17
 18      END LOOP;
 19
 20      COMMIT;
 21
 22      DBMS_OUTPUT.PUT_LINE('Discount Applied');
 23
 24  END;
 25  /
Discount Applied

PL/SQL procedure successfully completed.

SQL> SELECT * FROM Loans;

    LOANID CUSTOMERID INTERESTRATE DUEDATE
---------- ---------- ------------ ---------
       101          1            9 12-JUL-26
       102          2           12 11-AUG-26
       103          3           10 02-JUL-26
```

Scenario 2: A customer can be promoted to VIP status based on their balance.
Question: Write a PL/SQL block that iterates through all customers and sets a flag IsVIP to TRUE for those with a balance over $10,000.

```
SQL> BEGIN
  2
  3      FOR customer_rec IN
  4      (
  5          SELECT CustomerID, Balance
  6          FROM Customers
  7      )
  8      LOOP
  9
 10          IF customer_rec.Balance > 10000 THEN
 11
 12              UPDATE Customers
 13              SET IsVIP = 'TRUE'
 14              WHERE CustomerID = customer_rec.CustomerID;
 15
 16          END IF;
 17
 18      END LOOP;
 19
 20      COMMIT;
 21
 22      DBMS_OUTPUT.PUT_LINE('VIP Status Updated');
 23
 24  END;
 25  /
VIP Status Updated

PL/SQL procedure successfully completed.

SQL> SELECT * FROM Customers;

CUSTOMERID CUSTOMERNAME                                       AGE    BALANCE ISVIP
---------- -------------------------------------------------- ---------- ---------- -----
         1 Sai                                                 65      15000 TRUE
         2 Ravi                                                45       8000 FALSE
         3 Priya                                               70      20000 TRUE
```

Scenario 3: The bank wants to send reminders to customers whose loans are due within the next 30 days.

Question: Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer.

```
SQL> BEGIN
  2
  3      FOR loan_rec IN
  4      (
  5          SELECT c.CustomerName,
  6                 l.LoanID,
  7                 l.DueDate
  8          FROM Customers c
  9          JOIN Loans l
 10          ON c.CustomerID = l.CustomerID
 11          WHERE l.DueDate BETWEEN SYSDATE
 12                              AND SYSDATE + 30
 13      )
 14      LOOP
 15
 16          DBMS_OUTPUT.PUT_LINE(
 17              'Reminder: '
 18              || loan_rec.CustomerName
 19              || ', your loan '
 20              || loan_rec.LoanID
 21              || ' is due on '
 22              || TO_CHAR(loan_rec.DueDate,'DD-MON-YYYY')
 23          );
 24
 25      END LOOP;
 26
 27  END;
 28  /
Reminder: Sai, your loan 101 is due on 12-JUL-2026
Reminder: Priya, your loan 103 is due on 02-JUL-2026

PL/SQL procedure successfully completed.

SQL>
```

```
SQL> SELECT * FROM Employees;

EMPLOYEEID EMPLOYEENAME    DEPARTMENT  SALARY
---------- --------------- ---------- -------
         1 John            IT           50000
         2 David           IT           60000
         3 Mary            HR           45000

SQL> SELECT * FROM Accounts;

 ACCOUNTID    BALANCE
---------- ----------
      1001       5000
      1002       3000
```

Exercise 3: Stored Procedures

Scenario 1: The bank needs to process monthly interest for all savings accounts.
Question: Write a stored procedure **ProcessMonthlyInterest** that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

```
SQL> CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
  2  IS
  3  BEGIN
  4
  5      UPDATE Accounts
  6      SET Balance = Balance + (Balance * 0.01);
  7
  8      COMMIT;
  9
 10      DBMS_OUTPUT.PUT_LINE('Monthly Interest Processed');
 11
 12  END;
 13  /

Procedure created.

SQL> EXEC ProcessMonthlyInterest;
Monthly Interest Processed

PL/SQL procedure successfully completed.

SQL>
SQL> SELECT * FROM Accounts;

 ACCOUNTID    BALANCE
---------- ----------
      1001       5050
      1002       3030
```

Scenario 2: The bank wants to implement a bonus scheme for employees based on their performance.
Question: Write a stored procedure **UpdateEmployeeBonus** that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

```
SQL> CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus
  2  (
  3      p_department IN VARCHAR2,
  4      p_bonus_percent IN NUMBER
  5  )
  6  IS
  7  BEGIN
  8
  9      UPDATE Employees
 10      SET Salary = Salary + (Salary * p_bonus_percent / 100)
 11      WHERE Department = p_department;
 12
 13      COMMIT;
 14
 15      DBMS_OUTPUT.PUT_LINE('Bonus Updated');
 16
 17  END;
 18  /

Procedure created.

SQL> EXEC UpdateEmployeeBonus('IT',10);
Bonus Updated

PL/SQL procedure successfully completed.
```

Scenario 3: Customers should be able to transfer funds between their accounts.
Question: Write a stored procedure **TransferFunds** that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

```
SQL> CREATE OR REPLACE PROCEDURE TransferFunds
  2  (
  3      p_source_account      IN NUMBER,
  4      p_destination_account IN NUMBER,
  5      p_amount              IN NUMBER
  6  )
  7  IS
  8      v_balance NUMBER;
  9  BEGIN
 10
 11      SELECT Balance
 12      INTO v_balance
 13      FROM Accounts
 14      WHERE AccountID = p_source_account;
 15
 16      IF v_balance >= p_amount THEN
 17
 18          UPDATE Accounts
 19          SET Balance = Balance - p_amount
 20          WHERE AccountID = p_source_account;
 21
 22          UPDATE Accounts
 23          SET Balance = Balance + p_amount
 24          WHERE AccountID = p_destination_account;
 25
 26          COMMIT;
 27
 28          DBMS_OUTPUT.PUT_LINE('Transfer Successful');
 29
 30      ELSE
 31
 32          DBMS_OUTPUT.PUT_LINE('Insufficient Balance');
 33
 34      END IF;
 35
 36  END;
 37  /

Procedure created.
```

```
SQL> EXEC TransferFunds(1001,1002,1000);
Transfer Successful

PL/SQL procedure successfully completed.

SQL> SELECT * FROM Accounts;

 ACCOUNTID    BALANCE
---------- ----------
      1001       4050
      1002       4030

SQL>
```